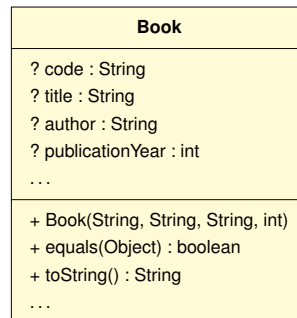


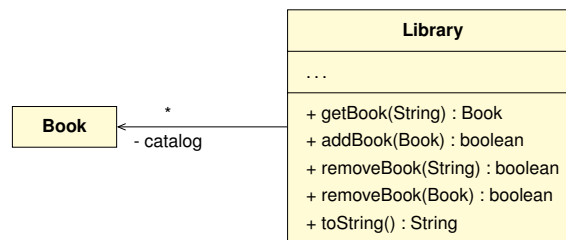
Exercice 1 : Gestion d'un fond documentaire

On souhaite développer un outil de gestion du fond documentaire d'une bibliothèque. Les livres sont caractérisés par une cote (un code d'identification), un titre, un auteur et une année de parution.

Q1. Après avoir bien réfléchi à l'accessibilité et la visibilité des attributs, écrivez une classe `Book` munie des accesseurs et des mutateurs pertinents, d'un constructeur complètement paramétré et d'une méthode `toString()`.



Q2. Écrivez une classe `Library` contenant un ensemble de livre. Pour des raisons pratiques, on souhaite pouvoir faire des recherches par rapport aux cotes des livres. On souhaite pouvoir accéder aux livres à partir de leur cote, ajouter de nouveaux livres, en supprimer soit à partir de leur cote, soit en fournissant directement l'ouvrage. On doit enfin pouvoir afficher la liste des livres disponibles dans la bibliothèque.



Les signatures à respecter sont les suivantes :

```

Book getBook(String code)
boolean addBook(Book b)
boolean removeBook(String aCode)
boolean removeBook(Book b)
String toString()
    
```

Q3. Écrivez une classe `UseLibrary` qui réalise le scénario suivant : la création de 3 livres et leur ajout à la bibliothèque, la recherche d'un livre selon sa cote et l'affichage du catalogue de la bibliothèque. L'affichage obtenu doit être le suivant :

A specific book:

```
Book [H2G2:The Hitchhiker's Guide to the Galaxy->D. Adams,1979]
```

Display of the library

```
[Book [H2G2:The Hitchhiker's Guide to the Galaxy->D. Adams,1979], Book
```

```
→ [FLTL:Flatland->E.Abbott Abbott,1884], Book [REU:The Restaurant at the End of the
```

```
→ Universe->D. Adams,1980]]
```

On souhaite désormais gérer les emprunts des livres. Pour simplifier, les emprunteurs seront réduits à un numéro d'identification entier. On doit pouvoir déterminer si un livre est emprunté et par qui le cas échéant.

Q4. Ajoutez à la classe `Book` les méthodes `borrow()` permettant à un utilisateur donné d'emprunter un livre (si possible), `giveBack()` pour rendre un livre (retournant un booléen représentant l'éventuel succès de la méthode ou non) et `isAvailable()` déterminant si un livre est libre ou non. Ajoutez un attribut `borrower` entier, représentant l'identifiant de l'emprunteur, qui vaut 0 si le livre est libre ou l'identifiant de l'emprunteur sinon. Les signatures à satisfaire sont les suivantes :

```
boolean borrow(int borrower)
boolean giveBack()
boolean isAvailable()
```

Q5. Ajoutez à la classe `Library` les méthodes `borrowings()` qui établit la liste textuelle des emprunts en cours, `borrow()` qui permet d'emprunter un livre donné, et `giveBack()` qui permet de rendre un livre. Les signatures à respecter sont les suivantes :

```
String borrowings()
boolean borrow(String code, int borrower)
boolean giveBack(String code)
```

Q6. Écrivez un scénario dans la classe `UseLibrary`, basé sur deux emprunteurs (de numéro 42 et 404 dans l'exemple suivant) montrant l'évolution de la liste des emprunts après une tentative fructueuse (de 42), une infructueuse (de 404, car le livre est déjà emprunté par 42), une tentative fructueuse (de 404 sur un autre ouvrage) et enfin une dernière fructueuse (de 42). On doit obtenir l'affichage suivant :

```
Borrowings list
Initially -->
true --> (H2G2)--42
false--> (H2G2)--42
true--> (H2G2)--42 (REU)--404
true --> (H2G2)--42 (FLTL)--42 (REU)--404
```

On souhaite gérer de manière particulière les bandes dessinées. Les bandes dessinées sont des livres, mais qui sont en plus caractérisées par un illustrateur. On doit pouvoir les manipuler comme n'importe quel livre, tout en les affichant de manière spécifique.

Q7. Créez une classe `ComicBook` répondant à ces besoins. La signature du constructeur est la suivante :

```
ComicBook(String code, String title, String author, int publicationYear, String illustrator)
```

Q8. Écrivez un nouveau scénario dans `UseLibrary`, montrant qu'on peut ajouter et emprunter des bandes dessinées de la même manière que des livres. On obtient l'affichage suivant :

```
> [Book [H2G2:The Hitchhiker's Guide to the Galaxy->D. Adams,1979],
  ↳ ComicBook[LeuG:Léonard est un Génie->Bob de Groot,1977,Turk]]
true--> (H2G2)--42
true --> (H2G2)--42 (LeuG)--42
```

Q9. Écrivez les méthodes `stockSize` et `borrowedBookNumber` qui retournent respectivement le nombre de livres référencés dans le catalogue et le nombre de livres en cours d'emprunt.

Q10. Vérifiez la validation du troisième scénario de la classe de tests `LibraryTest`.

On souhaite enfin gérer la durée des emprunts et les retardataires. Les livres et les bandes dessinées ont des durées d'emprunt maximales différentes. La durée maximale d'emprunt pour un livre est toujours de 15 jours. Pour une bande dessinée, cette durée maximale d'emprunt est réduite à 5 jours si la bande dessinée est parue il y a 2 ans ou moins, sinon elle est traitée comme un livre normal.

Q11. Ajoutez un attribut `borrowingDate` à vos objets `Book` et `ComicBook`, ainsi qu'une méthode retournant la durée maximale d'emprunt selon la nature de la ressource (`getDurationMax()`), et une méthode qui retourne la date

avant laquelle il faut rendre la ressource (`getGiveBackDate()`). Adaptez si nécessaire les méthodes d'emprunt et de restitution.

Q12. Écrivez un scénario dans `UseLibrary` montrant qu'on peut emprunter indifféremment des livres et des bandes dessinées, mais que les dates de retour ne sont pas les mêmes. Il faut obtenir l'affichage suivant :

```
> [Book [H2G2:The Hitchhiker's Guide to the Galaxy->D. Adams,1979],
  ↳ ComicBook[LeuG:Léonard est un Génie->Bob de Groot,1977,Turk], ComicBook[GJV:Génie,
  ↳ Vidi, Vici !->Turk,2019,Zidrou]]
2022-04-01
2022-04-01
2022-04-01
```

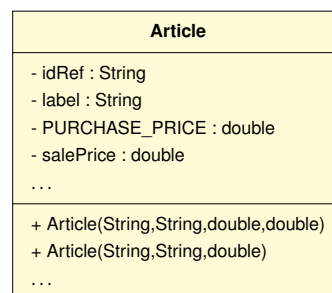
Q13. Récupérez la classe de tests `LibraryTest` et intégrez-la à votre projet. Vérifiez la validation de tous les scénarios.

Q14. Réalisez un `commit` associé au message : "tp00-05::exo-biblio".

Exercice 2 : Gestion d'inventaires dans un magasin

Dans un magasin, on gère une liste d'articles plus ou moins rentables. On ne gère pas de la même manière des articles périssables et des articles non périssables, même s'ils ont un grand nombre de caractéristiques en commun.

Q1. Écrivez une classe `Article` caractérisée par une *référence*, un *label*, un *prix d'achat* et un *prix de vente*. Le prix d'achat n'est plus modifiable une fois une instance créée. Par ailleurs, on doit être en mesure de créer un article sans spécifier son prix de vente. Dans un tel cas, chaque magasin fixe une marge de bénéfice à faire sur tous les articles (par exemple, 0.2 représente 20% de marge).



```
Article(String idRef, String label, double purchasePrice, double salePrice)
```

```
Article(String idRef, String label, double purchasePrice)
```

Q2. Munissez cette classe d'un accesseur et d'un mutateur pour l'attribut `salePrice`, d'un accesseur uniquement pour l'attribut constant `PURCHASE_PRICE`, et d'une méthode `getMargin()` qui sert à déterminer la marge de profit brut réalisée sur cet article.

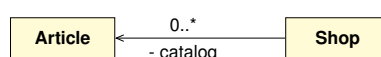
```
double getMargin()
```

Q3. Adjoignez une méthode `toString()` permettant de retourner un article sous forme textuelle, permettant l'affichage suivant pour une *chaise* de référence `42err404` achetée 20 € :

```
Article a = new Article("42err404", "chaise", 20.00);
System.out.println(a);
```

```
Article [42err404:chaise=20.0e/24.0e]
```

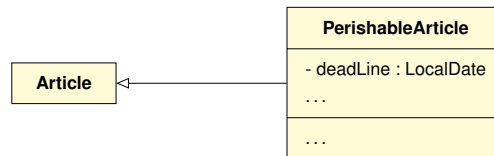
Q4. Considérons maintenant un magasin qui maintient une liste de références d'articles qu'il gère. Créez la classe `Shop` satisfaisant ces spécifications et munie d'une méthode `toString()` permettant la représentation textuelle des références à raison d'un article par ligne. On se contentera du constructeur par défaut pour cette classe.



Q5. Munissez cette classe d'une méthode `addArticle()` permettant l'ajout d'un article ou d'un ensemble d'articles donnés en paramètre.

Parmi les produits, certains sont périssables et doivent donc être gérés de manière particulière. Ils sont notamment caractérisés par une date de péremption. Ils perdent toute valeur marchande dès que cette date est dépassée, ils sont donc le plus souvent bradés à l'approche de l'échéance pour éviter de les perdre complètement.

Q6. Écrivez une classe `PerishableArticle` sachant qu'on doit pouvoir créer une instance sans prix de vente (comme pour `Article` la marge de bénéfice par défaut est fixé à 20%) ou sans date limite (elle est fixée par défaut à 10 jours de la date courante).



```
PerishableArticle(String idRef, String label, double pA, double pV, LocalDate dl)
PerishableArticle(String idRef, String label, double pA, LocalDate dl)
PerishableArticle(String idRef, String label, double pA, double pV)
PerishableArticle(String idRef, String label, double pA)
```

Q7. Adjoignez un accesseur pour l'attribut `deadLine` ainsi qu'une méthode de représentation textuelle, complétant l'affichage d'un article standard et permettant l'affichage sous la forme :

```
PerishableArticle ap = new PerishableArticle("42err404", "milk", 20.00);
System.out.println(ap);
```

```
PerishableArticle [2err404:milk=20.0€/24.0€-->2022-01-26]
```

Q8. Écrivez une classe `UseArticle` munie d'une méthode décrivant un scénario où l'on crée cinq articles dont deux périssables, qu'on ajoute au magasin (au sein bien sûr de la même structure de données) avant de l'afficher.

Q9. On souhaite désormais pouvoir faire des traitements qui ne doivent concerner que les articles périssables. Écrivez une méthode `getPerishables()` qui retourne tous les articles périssables du magasin. Une méthode support `isPerishable` mène à une bonne décomposition du code.

```
List<PerishableArticle> getPerishables()
```

Q10. Écrivez deux méthodes déterminant respectivement le nombre d'articles et le nombre d'articles périssables dans le catalogue. Les signatures à respecter sont les suivantes :

```
int getNbArticle()
int getNbPerishableArticle()
```

Q11. On souhaite faire des démarques sur certains produits périssables. Écrivez `discountPerishable()`, une méthode qui applique un rabais d'un taux donné à tous les produits périssables dont la date limite arrive à échéance avant un seuil donné.

```
void discountPerishable(LocalDate threshold, double rate)
```

Q12. Écrivez dans votre classe `UseArticle` un scénario, similaire au précédent, validant la démarque basée sur une date des produits périssables (démarque qui ne doit concerner qu'un seul des deux produits périssables de la liste).

Q13. Sur le même modèle, on souhaite pouvoir gérer les périodes de soldes, qui ne concerne que les produits non périssables. Fournissez à la classe `Shop` une méthode `discountNotPerishable()` permettant d'appliquer un rabais d'un taux donné à tous les produits non périssables.

```
void discountNotPerishable(double rate)
```

Q14. Récupérez la classe de tests `ShopTest` et validez le bon fonctionnement de vos méthodes.

Q15. On souhaite enfin pouvoir identifier quel est le produit le plus rentable (quelle que soit sa nature). Écrivez la méthode nécessaire pour cela.

```
Article mostProfitable()
```

Q16. Réalisez un `commit` associé au message : `"tp00-05::exo-périssables"`. Soumettez ensuite l'ensemble des `commit` au dépôt.